

TÀI LIỆU ĐẶC TẢ YÊU CẦU PHẦN MỀM (SOFTWARE REQUIREMENTS SPECIFICATION)

DỰ ÁN: TRUNG TÂM VẬN HÀNH LOGISTICS THÔNG MINH TÍCH HỢP AI (SMARTHUB)

THÔNG TIN QUẢN TRỊ TÀI LIỆU (DOCUMENT CONTROL)

Thuộc tính	Chi tiết
Tên dự án	SmartHub — AI-Powered Smart Logistics Operations Center
Mã tài liệu	SRS-AI-SMARTHUB-V1.0
Môn học	AI Integrated in Action (AI_PTIT_K24)
Phiên bản	1.0.0 (Release Candidate)
Trạng thái	Official Research Specification
Nền tảng công nghệ	Java • Spring Boot • Spring AI • Supabase (Pgvector) • MCP • Langfuse

1. GIỚI THIỆU TỔNG QUAN (INTRODUCTION & OBJECTIVES)

1.1. Mục đích tài liệu (Purpose)

Tài liệu Đặc tả Yêu cầu Phần mềm (SRS) này thiết lập các yêu cầu chức năng, yêu cầu phi chức năng, kiến trúc kỹ thuật và định hướng nghiên cứu chuyên sâu cho hệ thống **SmartHub** — Trung tâm điều hành và hỗ trợ vận tải thông minh tích hợp trí tuệ nhân tạo (AI) phục vụ doanh nghiệp logistics **RikkeiExpress**.

1.2. Bối cảnh nghiệp vụ & Phạm vi hệ thống (Business Context & System Scope)

Trong bối cảnh lưu lượng vận chuyển hàng hóa tăng trưởng mạnh, doanh nghiệp đối mặt với 3 bài toán lớn cần tối ưu hóa bằng AI:

3 BÀI TOÁN NGHIỆP VỤ CẦN TỐI ƯU HÓA	
Bài toán nghiệp vụ	Thực trạng & Vấn đề tồn đọng
1. Quá tải tra cứu quy chế	Khách hàng và nhân viên bưu cục mất nhiều thời gian tra cứu thủ công trong tập tài liệu PDF.
2. Độ trễ điều phối sự cố	Tiếp nhận khiếu nại và cập nhật trạng thái đơn hàng thủ công làm chậm quy trình giải quyết.
3. Rủi ro quản trị & Chi phí AI	Thiếu khả năng giám sát vết chạy (Tracing) và kiểm soát chi phí token khi triển khai LLM.

- **Phạm vi triển khai (In-Scope):**
 - Xây dựng Core Backend bằng Java Spring Boot tích hợp Spring AI.
 - Cung cấp các REST API cho 4 phân hệ: Tra cứu tri thức (RAG), Điều phối sự cố tự động (Agent), Phân tích dữ liệu qua giao thức Model Context Protocol (MCP), và Giám sát vận hành (Observability với Langfuse).
 - Quản trị dữ liệu quan hệ và dữ liệu vector trên PostgreSQL (Supabase / Local Pgvector).
- **Giới hạn phạm vi (Out-of-Scope):**
 - Không xây dựng ứng dụng di động (Mobile App) hay cổng thanh toán trực tuyến.
 - Không can thiệp vào giai đoạn tiền huấn luyện (Pre-training) của các mô hình nền tảng.

1.3. Định nghĩa và Thuật ngữ viết tắt (Glossary & Definitions)

Thuật ngữ	Tên viết tắt	Định nghĩa nghiệp vụ & kỹ thuật
Retrieval-Augmented Generation	RAG	Kỹ thuật truy xuất thông tin từ kho tri thức vector ngoài để bổ sung ngữ cảnh cho LLM.
Model Context Protocol	MCP	Giao thức mở chuẩn hóa cách thức AI Client kết nối an toàn với Tools và Data Sources bên ngoài.

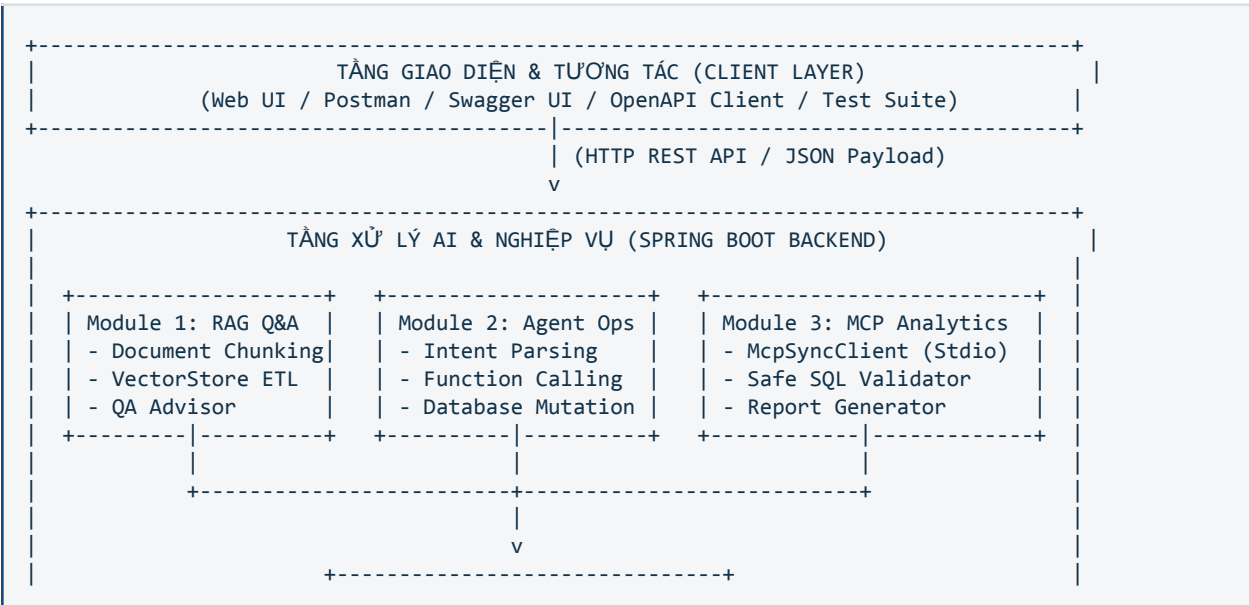
Function Calling / Tools	Tools	Cơ chế cho phép LLM quyết định gọi hàm Java để tương tác CSDL hoặc hệ thống ngoài.
Observability & Tracing	LLMOps	Hệ thống giám sát vết chạy phân cấp (Traces/Spans), độ trễ, số token và chi phí gọi LLM.
Vector Similarity Search	Pgvector	Extension mở rộng của PostgreSQL hỗ trợ lưu trữ và tìm kiếm tương đồng vector (Cosine Distance).

1.4. Mục tiêu nghiên cứu trọng tâm (Research Objectives)

1. RQ-01 (Semantic Search Accuracy): Nghiên cứu tối ưu hóa chiến lược phân mảnh tài liệu (Chunking) và ngưỡng tương đồng (Similarity Threshold) để nâng cao độ chính xác của RAG.
2. RQ-02 (Agentic Autonomous Workflow): Nghiên cứu khả năng bóc tách thực thể và cơ chế lập trình phòng thủ khi AI Agent tự động gọi Tool cập nhật CSDL.
3. RQ-03 (Standardized Protocol Integration): Đánh giá hiệu quả bảo mật, chống Stdio Pollution và ngăn chặn SQL Injection khi tích hợp MCP Server.
4. RQ-04 (Cost & Latency Optimization): Thiết lập hệ thống đo lường thời gian thực, đánh giá chi phí token và ngăn chặn rủi ro vòng lặp vô hạn.

2. KIẾN TRÚC HỆ THỐNG TỔNG THỂ (SYSTEM ARCHITECTURE)

2.1. Sơ đồ phân tầng kiến trúc (Layered Architecture Diagram)





2.2. Danh mục công nghệ chuẩn hóa (Technology Stack)

Thành phần	Công nghệ lựa chọn	Mục đích sử dụng
Ngôn ngữ nền tảng	Java 17 / 21 LTS	Nền tảng thực thi backend doanh nghiệp
Backend Framework	Spring Boot 3.3.x	Framework xây dựng dịch vụ RESTful
AI Integration	Spring AI Framework	Khung tích hợp ChatClient, VectorStore, Advisors & MCP
Mô hình LLM	OpenAI gpt-4o-mini / Gemini 1.5-flash	Xử lý ngôn ngữ tự nhiên, lập luận và sinh câu trả lời
Mô hình Embedding	OpenAI text-embedding-3-small	Chuyển đổi văn bản thành vector ngữ nghĩa 1536 chiều
Cơ sở dữ liệu	PostgreSQL + Pgvector Extension	Lưu trữ hỗn hợp dữ liệu quan hệ và vector nhúng
Giao thức mở rộng	Model Context Protocol (Stdio)	Kết nối AI Client với MCP Server nội bộ
LLMOps Observability	Langfuse Platform / OpenTelemetry	Thu thập và phân tích Trace, Span, Latency, Token Usage

3. ĐẶC TẢ YÊU CẦU CHỨC NĂNG & CHUYÊN ĐỀ NGHIÊN CỨU (FUNCTIONAL MODULES)

Hệ thống bao gồm 4 phân hệ nghiên cứu trọng tâm:

TỔNG QUAN 4 PHÂN HỆ NGHIÊN CỨU	
Phân hệ	Nội dung bài toán nghiên cứu & đặc tả cốt lõi
Phân hệ 1 (RAG)	Tra cứu tri thức quy chế, chiến lược Chunking & Vector DB
Phân hệ 2 (Agent)	Function Calling, trích xuất tham số & cập nhật CSDL tự động
Phân hệ 3 (MCP)	Tích hợp Model Context Protocol & phòng vệ câu lệnh SQL
Phân hệ 4 (LLMOps)	Giám sát Observability, trực quan hóa Trace & tối ưu Token

3.1. Chuyên đề 1: Nghiên cứu Hệ thống RAG Tra Cứu Quy Chế Vận Chuyển

- Bài toán thực tiễn:

Tài liệu quy chế logistics chứa nhiều điều khoản bồi thường, biểu phí theo trọng lượng và thời gian giao cam kết. Làm sao để xây dựng hệ thống hỏi đáp tự động giúp người dùng tra cứu chính xác, trích dẫn đúng số điều/khoản mà không bị bịa đặt thông tin?

- Các câu hỏi gợi mở để nghiên cứu & thử nghiệm:

- Nghiên cứu về Chiến lược Chunking: Kích thước đoạn (Chunk Size: 300 vs 500 vs 1000 ký tự) và độ chồng lặp (Overlap: 0% vs 10% vs 20%) ảnh hưởng như thế nào đến độ chính xác khi tìm kiếm ngữ cảnh?

– *Nghiên cứu về Thuật toán tìm kiếm tương đồng*: Khoảng cách Cosine Similarity hoạt động ra sao trong Pgvector? Đặt ngưỡng tương đồng (Similarity Threshold) bao nhiêu để loại bỏ các đoạn văn bản không liên quan?

– *Nghiên cứu về Prompting cho RAG*: Thiết kế System Prompt như thế nào để yêu cầu AI **bắt buộc trích dẫn nguồn** và thừa nhận "Tôi không tìm thấy thông tin trong tài liệu" nếu dữ liệu không có sẵn?

- **Yêu cầu & Tiêu chuẩn cần đạt sau khi hoàn thành (Target Standards):**

– Nạp thành công tài liệu quy chế (PDF/Markdown dài tối thiểu 3 trang A4) vào bảng **vector_store** trên Pgvector/PostgreSQL qua cơ chế Chunking & Embedding.

– Xây dựng API **GET /api/v1/rag/ask?question=...** tích hợp **QuestionAnswerAdvisor** trả lời chính xác câu hỏi tra cứu chính sách theo nội dung tài liệu.

– Phản hồi bắt buộc phải có trích dẫn nguồn cụ thể (**sourceDocuments** gồm tên tài liệu, số trang hoặc số điều/khoản).

– Tiêu chuẩn chống ảo tưởng (Anti-Hallucination): Tự động từ chối trả lời lịch sự nếu thông tin không có trong tài liệu quy chế.

- **Sản phẩm đầu ra kỳ vọng:**

– Pipeline nạp tài liệu vào CSDL vector.

– API tra cứu **GET /api/v1/rag/ask?question=...** trả về câu trả lời kèm trích dẫn nguồn tài liệu gốc.

3.2. Chuyên đề 2: Nghiên cứu Trợ Lý Vận Hành Tự Động Xử Lý Sự Cố (Agent & Function Calling)

- **Bài toán thực tiễn:**

Khách hàng phản ánh sự cố bằng ngôn ngữ tự nhiên lộn xộn (ví dụ: "*Đơn RK-2026-001 của tôi gửi tuần trước bị ướt sũng hỏng đồ ở kho Hà Nội, đề nghị kiểm tra*"). Làm thế nào để AI Agent tự động bóc tách đúng các thông tin then chốt và thực thi cập nhật CSDL mà không cần con người thao tác tay?

- **Các câu hỏi gợi mở để nghiên cứu & thử nghiệm:**

– *Nghiên cứu về Trích xuất thực thể (Parameter Extraction)*: Làm thế nào để LLM phân biệt được mã vận đơn, loại sự cố (**HỎNG_HÓC**, **GIAO_TRỄ**), mức độ nghiêm trọng (**CRITICAL**, **LOW**) từ câu văn tự nhiên?

– *Nghiên cứu về Thiết kế Java Tools*: Định nghĩa mô tả công cụ (**@Description**) và cấu trúc DTO đầu vào như thế nào để LLM luôn sinh đúng kiểu dữ liệu JSON Schema?

– *Nghiên cứu về Lập trình phòng thủ (Defensive Programming)*: Điều gì xảy ra nếu LLM truyền mã đơn hàng không tồn tại trong CSDL? Tool Java cần phản hồi cấu trúc lỗi như thế nào để Agent hiểu và thông báo lại cho người dùng thay vì làm sập ứng dụng?

- **Yêu cầu & Tiêu chuẩn cần đạt sau khi hoàn thành (Target Standards):**

– Xây dựng API **POST /api/v1/operations/chat** tiếp nhận tin nhắn ngôn ngữ tự nhiên bất cấu trúc.

– Agent phải bóc tách chính xác 4 thực thể cốt lõi: Mã vận đơn (**trackingCode**), Loại sự cố (**incidentType**: **HỎNG_HÓC**, **GIAO_TRỄ**, **THẤT_LẠC**), Bưu cục (**hubCode**), Mức độ nghiêm trọng (**severity**: **LOW**, **MEDIUM**, **CRITICAL**).

- Tự động kích hoạt `createIncidentTool` tạo bản ghi mới trong bảng `incidents` (trạng thái `OPEN`) và gọi `updateDeliveryStatusTool` cập nhật trạng thái đơn hàng trong bảng `deliveries` sang `DAMAGED` hoặc `DELAYED`.
- Tiêu chuẩn phòng vệ (Defensive Standard): Bắt ngoại lệ an toàn và phản hồi lịch sự khi mã đơn hàng không tồn tại hoặc dữ liệu thiếu trường bắt buộc (Zero Crash Policy).
- Sản phẩm đầu ra kỳ vọng:
 - Các Java Function Tools (ví dụ: `createIncidentTool`, `updateDeliveryStatusTool`).
 - API tiếp nhận `POST /api/v1/operations/chat` tự động nhận diện sự cố, ghi nhận phiếu sự cố vào CSDL và cập nhật trạng thái đơn hàng.

3.3. Chuyên đề 3: Nghiên cứu Ứng Dụng Model Context Protocol (MCP) Trong Đối Soát Dữ Liệu

- Bài toán thực tiễn:

Ban Giám Đốc cần yêu cầu AI phân tích hiệu suất của từng bưu cục (ví dụ: *"Tổng hợp số lượng đơn giao trễ của bưu cục HN-01 trong tuần qua"*). Thay vì lập trình hàng chục API chuyên biệt, làm sao để ứng dụng giao thức mở Model Context Protocol (MCP) giúp AI tự khám phá công cụ và phân tích dữ liệu?

- Các câu hỏi gợi mở để nghiên cứu & thử nghiệm:

- *Nghiên cứu so sánh kiến trúc*: Điểm khác biệt cốt lõi giữa việc gọi REST API thông thường và cơ chế bắt tay (Handshake/Capabilities Exchange) qua Stdio Transport của giao thức MCP là gì?
- *Nghiên cứu rủi ro Stdio Pollution*: Tại sao log console (`System.out.println`) lại làm hỏng đường truyền JSON-RPC của MCP? Phương án cấu hình log (`logback-spring.xml`) như thế nào để bảo vệ luồng truyền tin?
- *Nghiên cứu phòng vệ an toàn dữ liệu*: Làm thế nào để xây dựng lớp kiểm duyệt câu lệnh SQL (Safe SQL Validator) ngăn chặn triệt để các lệnh nguy hiểm (`DROP`, `DELETE`) khi AI tạo câu truy vấn?

- Yêu cầu & Tiêu chuẩn cần đạt sau khi hoàn thành (Target Standards):

- Xây dựng MCP Server chạy qua Stdio Transport kết nối trực tiếp với PostgreSQL, công khai các công cụ truy vấn dữ liệu bưu cục và xuất báo cáo Markdown.
- Spring Boot Client sử dụng `spring-ai-mcp-client` kết nối và tự động chuyển đổi MCP Tools thành Function Callbacks trong `ChatClient`.
- Tiêu chuẩn cách ly luồng Stdio: Cấu hình `logback-spring.xml` chuyển toàn bộ console log sang `System.err` và tắt Banner Spring Boot để triệt tiêu lỗi Stdio Pollution (không làm hỏng gói tin JSON-RPC).
- Tiêu chuẩn an toàn CSDL (Safe SQL): Tích hợp bộ lọc chỉ cho phép lệnh `SELECT`, tự động ép `LIMIT 100`, từ chối mọi câu lệnh phá hoại cấu trúc hoặc xóa dữ liệu (`DROP`, `DELETE`, `UPDATE`, `ALTER`).

- Sản phẩm đầu ra kỳ vọng:

- MCP Server độc lập cung cấp các công cụ truy vấn dữ liệu vận tải và xuất báo cáo Markdown.
- Spring Boot Client kết nối qua `McpSyncClient` và tích hợp mượt mà vào `ChatClient`.

3.4. Chuyên đề 4: Nghiên cứu Hệ Thống Đo Lường & Đánh Giá Chi Phí LLMOps (Langfuse Observability)

- **Bài toán thực tiễn:**

Một ứng dụng AI khi triển khai vào doanh nghiệp không thể là một "hộp đen" (Black-box). Cần có cơ chế đo lường xem mỗi lượt gọi tốn bao nhiêu thời gian, tiêu thụ bao nhiêu token và chi phí thực tế là bao nhiêu USD.
- **Các câu hỏi gợi mở để nghiên cứu & thử nghiệm:**
 - *Nghiên cứu về Cây phân cấp vết chạy (Trace & Span Hierarchy):* Làm thế nào để trực quan hóa một chu trình phức tạp (từ lúc Client gửi request -> gọi LLM -> kích hoạt Tool 1 -> cập nhật DB -> trả kết quả) trên giao diện Langfuse?
 - *Nghiên cứu về Tối ưu hóa chi phí (Cost Optimization):* So sánh mức độ tiêu thụ token giữa việc đưa toàn bộ dữ liệu vào Prompt (In-Context) so với việc dùng RAG/Tools. Giải pháp nào tiết kiệm chi phí hơn?
 - *Nghiên cứu về Phòng chống vòng lặp vô hạn (Infinite Loop Guard):* Cấu hình giới hạn số lượt gọi công cụ (Max Iterations) như thế nào để ngăn chặn tình trạng Agent lặp vô tận gây tốn chi phí API?
- **Yêu cầu & Tiêu chuẩn cần đạt sau khi hoàn thành (Target Standards):**
 - Tích hợp Langfuse SDK / OpenTelemetry Exporter vào Spring Boot backend để tự động thu thập Telemetry cho toàn bộ các lượt gọi LLM và Tool Execution.
 - Trực quan hóa cây vết chạy phân cấp (Trace & Span Hierarchy) hiển thị rõ thời gian thực thi (Latency) của từng bước xử lý.
 - Thống kê chính xác số lượng `prompt_tokens`, `completion_tokens`, tổng tokens và ước tính chi phí USD thực tế trên Langfuse Dashboard.
 - Tiêu chuẩn kiểm soát an toàn vận hành: Cấu hình giới hạn số lượt gọi công cụ tối đa (`max-iterations <= 6`) để ngăn chặn hoàn toàn rủi ro Agent rơi vào vòng lặp vô hạn.
- **Sản phẩm đầu ra kỳ vọng:**
 - Dashboard Langfuse ghi nhận đầy đủ vết chạy thực tế của cả 3 chuyên đề trên.
 - Báo cáo thống kê phân tích thời gian phản hồi (Latency) và chi phí token trong quá trình thử nghiệm.

4. ĐẶC TẢ CƠ SỞ DỮ LIỆU (DATABASE SPECIFICATIONS & DATA DICTIONARY)

Hệ thống sử dụng cơ sở dữ liệu PostgreSQL tích hợp tiện ích mở rộng `pgvector`. Dưới đây là đặc tả chi tiết cấu trúc các bảng:

4.1. Bảng `deliveries` (Quản lý Thông tin Đơn hàng & Vận chuyển)

Bảng lưu trữ thông tin thực tế của các đơn hàng trong mạng lưới bưu cục phục vụ đối soát và cập nhật trạng thái.

Tên cột (Column Name)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraints)	Ý nghĩa nghiệp vụ (Description)
-----------------------	--------------------------	-------------------------	---------------------------------

id	BIGSERIAL	PRIMARY KEY	Khóa chính tự tăng của bản ghi đơn hàng
tracking_code	VARCHAR(50)	UNIQUE, NOT NULL	Mã vận đơn định danh duy nhất (Ví dụ: RK-2026-001)
customer_name	VARCHAR(100)	NOT NULL	Họ tên khách hàng gửi hoặc nhận
hub_code	VARCHAR(20)	NOT NULL	Mã bưu cục/kho xử lý (Ví dụ: HN-01, SG-02, DN-03)
status	VARCHAR(30)	NOT NULL	Trạng thái giao hàng (IN_TRANSIT, DELIVERED, DELAYED, DAMAGED)
cod_amount	DECIMAL(12,2)	DEFAULT 0	Số tiền thu hộ COD của đơn hàng (VNĐ)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm đơn hàng được tạo trên hệ thống

4.2. Bảng `incidents` (Quản lý Phiếu Sự cố Vận hành do Agent Khởi tạo)

Bảng ghi nhận tự động các sự cố phát sinh (hỏng hóc, giao trễ, thất lạc) khi AI Agent bóc tách từ tin nhắn phản ánh của khách hàng.

Tên cột (Column Name)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraints)	Ý nghĩa nghiệp vụ (Description)
id	BIGSERIAL	PRIMARY KEY	Khóa chính tự tăng của phiếu sự cố
tracking_code	VARCHAR(50)	NOT NULL	Mã vận đơn phát sinh sự cố
incident_type	VARCHAR(50)	NOT NULL	Phân loại lỗi sự cố (HỎNG_HÓC, GIAO_TRỄ, THẤT_LẠC)
hub_code	VARCHAR(20)	NOT NULL	Mã bưu cục xảy ra sự cố hoặc quản lý đơn
severity	VARCHAR(20)	NOT NULL	Mức độ nghiêm trọng (LOW, MEDIUM, CRITICAL)

description	TEXT	NOT NULL	Nội dung mô tả chi tiết sự cố do AI Agent bóc tách
status	VARCHAR(30)	DEFAULT 'OPEN'	Trạng thái xử lý phiếu (OPEN, IN_PROGRESS, RESOLVED)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Thời điểm hệ thống ghi nhận sự cố

4.3. Bảng `vector\_store` (Lưu trữ Vector & Ngữ cảnh Quy chế RAG)

Bảng lưu trữ các đoạn văn bản (chunks) và vector nhúng ngữ nghĩa của tài liệu quy chế phục vụ tìm kiếm tương đồng (Similarity Search).

Tên cột (Column Name)	Kiểu dữ liệu (Data Type)	Ràng buộc (Constraints)	Ý nghĩa nghiệp vụ (Description)
id	UUID / BIGSERIAL	PRIMARY KEY	Khóa chính định danh đoạn văn bản (Chunk ID)
content	TEXT	NOT NULL	Nội dung văn bản quy chế sau khi phân mảnh (Chunk text)
metadata	JSON / JSONB	DEFAULT '{}'	Metadata kèm theo (Tên tài liệu, số trang, số điều khoản trích dẫn)
embedding	vector(1536)	NOT NULL	Vector nhúng ngữ nghĩa sinh bởi mô hình Embedding

5. YÊU CẦU PHI CHỨC NĂNG & TIÊU CHUẨN KỸ THUẬT (NON-FUNCTIONAL SPECIFICATIONS)

5.1. Hiệu năng & Giới hạn Độ trễ (Performance & Latency SLAs)

- Thời gian phản hồi truy vấn RAG: Dưới 3.0 giây đối với các câu hỏi tra cứu chính sách thông thường.
- Thời gian phản hồi Agent gọi Tool: Dưới 5.0 giây đối với chu trình bóc tách thực thể và cập nhật CSDL.

- **Tối ưu hóa Token:** Cấu hình Prompt ngắn gọn, súc tích, loại bỏ các chỉ dẫn dư thừa để kiểm soát chi phí vận hành.

5.2. An toàn Bảo mật & Quản trị Dữ liệu (Security & Governance)

- **Quản lý Khóa Bí mật (Secret Management):** Tuyệt đối không hardcode API Keys trong mã nguồn hoặc commit lên Git Repository. Toàn bộ thông tin nhạy cảm phải được nạp thông qua Environment Variables hoặc tệp cấu hình bảo mật.
- **Kiểm soát Truy cập Dữ liệu:** Áp dụng nguyên tắc đặc quyền tối thiểu (Least Privilege). Cơ chế MCP Tool chỉ được cấp quyền đọc (**SELECT**) trên các bảng dữ liệu cho phép.
- **Bảo vệ Luồng Truyền tin Stdio:** Toàn bộ log của Spring Boot và hệ thống máy chủ phải được chuyển hướng sang **System.err** để giữ luồng **stdout** hoàn toàn sạch cho các gói tin JSON-RPC.

5.3. Độ tin cậy & Khả năng Phục hồi (Reliability & Fault Tolerance)

- **Chống Vòng lặp Vô hạn:** Khống chế số lượt gọi Tool tối đa trong một phiên làm việc (**max-iterations <= 6**).
- **Xử lý Ngoại lệ An toàn (Resilience):** Tất cả các thao tác I/O, gọi Tool hoặc kết nối mạng bên ngoài phải được bọc trong khối **try-catch**, đảm bảo trả về phản hồi thân thiện và không gây sập ứng dụng (Zero Crash Policy).

6. TIÊU CHÍ ĐÁNH GIÁ & NGHIỆM THU HỆ THỐNG (SYSTEM ACCEPTANCE MATRIX)

Tiêu chí	Mô tả nghiệm thu kỹ thuật	Trọng số
Khả năng tra cứu RAG (Module 1)	Ingestion tài liệu thành công; trả lời đúng chính sách kèm trích dẫn điều khoản; không ảo tưởng.	25%
Khả năng tự động hóa Agent (Module 2)	Nhận diện đúng ý định sự cố; trích xuất chính xác tham số; tự động ghi nhận phiếu và cập nhật CSDL.	25%
Tích hợp giao thức MCP (Module 3)	Kết nối Stdio Ổn định, không lỗi Stdio Pollution; kiểm duyệt SQL an toàn; sinh báo cáo Markdown.	25%
Giám sát & Quản trị LLMOps (Module 4)	Ghi nhận đầy đủ vết Trace trên Langfuse; hiển thị rõ Latency và Token Usage; có cơ chế ngắt lặp.	25%
TỔNG CỘNG	Mức độ hoàn thiện hệ thống và chất lượng báo cáo kỹ thuật	100%